

Chuleta — Unit Testing Angular con Vitest

1. Comandos

```
ng test          # lanza los tests en modo watch (se re-ejecutan al
guardar)
ng test --coverage  # lanza tests + genera informe de cobertura en /coverage
```

Para usar `--coverage` hay que instalar el paquete de cobertura primero:

```
npm install @vitest/coverage-v8 --save-dev
```

El informe de cobertura se abre en el navegador desde: coverage/ejercicio3_meals_arroyo/index.html

2. Estructura de un test

```
describe('NombreComponente o Servicio', () => {

  beforeEach(() => {
    // se ejecuta ANTES de cada it → entorno limpio
  });

  afterEach(() => vi.restoreAllMocks()); // restaurar spies

  it('descripción de lo que se prueba', () => {
    // Arrange → preparo los datos
    // Act      → ejecuto la acción
    // Assert   → compruebo con expect()
  });

});
```

Flujo de ejecución:

```
beforeEach → it1
beforeEach → it2
beforeEach → it3
```

Cada `it` empieza desde cero gracias al `beforeEach`.

3. Patrón AAA (Arrange / Act / Assert)

```
it('should add appointment', () => {  
  // Arrange  
  const app = makeApp(1);  
  
  // Act  
  service.addAppointment(app);  
  
  // Assert  
  expect(service.getAppointments()).toHaveLength(1);  
  expect(service.hasPendingChanges()).toBe(true);  
});
```

4. TestBed — Angular en miniatura

TestBed crea un módulo Angular aislado solo para el test. Sin él, Angular no sabe cómo inyectar dependencias.

Para servicios:

```
TestBed.configureTestingModule({  
  providers: [MiServicio, DependenciaServicio],  
});  
const service = TestBed.inject(MiServicio);
```

Para componentes standalone:

```
TestBed.configureTestingModule({  
  imports: [MiComponente], // standalone: imports, NO declarations  
  providers: [provideRouter([])], // si usa RouterOutlet o routerLink  
});  
const fixture = TestBed.createComponent(MiComponente);
```

standalone vs NgModule:

- Angular 19+ (proyecto actual) → `imports: [Componente]`
- Angular antiguo con NgModule → `declarations: [Componente]`

5. fixture — el contenedor del componente

```
const fixture = TestBed.createComponent(App);  
const comp = fixture.componentInstance; // la clase → propiedades y métodos
```

```
const el = fixture.nativeElement as HTMLElement; // el DOM → querySelector
```

Crear ≠ Renderizar

Después de `createComponent`, el HTML todavía NO existe en el DOM. Hay que llamar a `detectChanges()`.

6. fixture.detectChanges()

Dispara el ciclo de detección de cambios de Angular:

- Ejecuta `ngOnInit`
- Renderiza el HTML (`{{ }}`, `@if`, `@for`, `signals` en `template`)
- Actualiza bindings tras cambios

```
fixture.detectChanges(); // renderiza el HTML
const h1 = el.querySelector('h1');
expect(h1?.textContent).toBe('Hola');
```

Regla de oro:

¿Vas a hacer `querySelector` o leer el DOM? → `detectChanges()` antes. ¿Solo miras propiedades o llamas métodos de la clase? → no hace falta.

7. JSDOM — el navegador falso

Los tests NO abren Chrome ni Firefox. Usan **JSDOM**, un navegador simulado en memoria que entiende HTML, CSS y eventos del DOM.

- Permite `querySelector`, `.click()`, `.textContent`, etc.
- Por eso los tests son rápidos (milisegundos, no segundos)
- Limitación: no renderiza CSS visual, no tiene GPU

8. Tests async — await en it

Cuando el método que pruebas es `async`, el callback del `it` también debe ser `async`:

```
// ❌ MAL — no espera a que resuelva la Promise
it('should return meal', () => {
  const result = service.getRandomMeal(); // devuelve Promise, no IMyMeal
  expect(result.idMeal).toBe(1); // falla: result es una Promise, no un objeto
});

// ✅ BIEN
it('should return meal', async () => {
  const result = await service.getRandomMeal();
```

```
expect(result.idMeal).toBe(1);  
});
```

Para probar que una Promise **rechaza** (lanza error):

```
it('should throw on error', async () => {  
  vi.spyOn(globalThis, 'fetch').mockReturnValue(Promise.reject(new  
  Error('fail')));  
  await expect(service.getRandomMeal()).rejects.toThrow('fail');  
});
```

9. Matchers avanzados — expect.arrayContaining / objectContaining

Útiles cuando no quieres comprobar el objeto entero, solo que contiene ciertas propiedades:

```
// El array contiene al menos estos elementos (puede tener más)  
expect(spy).toHaveBeenCalled(  
  expect.arrayContaining([  
    expect.objectContaining({ id: 1 }),  
    expect.objectContaining({ id: 2, assignedDay: WeekDay.Monday }),  
  ])  
);  
  
// El objeto contiene al menos estas propiedades  
expect(result).toEqual(  
  expect.objectContaining({ idMeal: 52772, strMeal: 'Teriyaki' })  
);
```

10. Matchers de expect más usados

```
expect(valor).toBe(x)           // igualdad estricta (===)
expect(valor).toEqual(x)        // igualdad profunda (objetos/arrays)
expect(valor).toBeTruthy()      // cualquier valor verdadero
expect(valor).toBeFalsy()        // cualquier valor falso
expect(valor).toBeNull()        // null exacto
expect(valor).toBeUndefined()   // undefined
expect(array).toHaveLength(n)    // longitud del array
expect(array).toContain(item)    // el array contiene ese item
expect(fn).toThrow()            // la función lanza un error
expect(promise).rejects.toThrow() // la promise rechaza con error
expect(spy).toHaveBeenCalled()    // el spy fue llamado
expect(spy).toHaveBeenCalledTimes(n) // llamado exactamente n veces
expect(spy).toHaveBeenCalledWith(x) // llamado con estos argumentos
expect(spy).not().toHaveBeenCalled() // NO fue llamado
```

11. Test negativo

NO significa que el test falle — significa que prueba que algo **NO ocurre**.

```
it('should not render footer', () => {
  const footer = el.querySelector('footer');
  expect(footer).toBeNull(); // comprobamos ausencia
});

it('should not save when no changes', () => {
  const spy = vi.spyOn(service, 'saveChanges');
  // no llamamos a nada que guarde
  expect(spy).not().toHaveBeenCalled();
});
```

12. Probar @Input()

```
fixture.componentRef.setInput('nombreInput', valor);
fixture.detectChanges(); // actualiza el HTML con el nuevo valor
expect(el.querySelector('p')?.textContent).toContain(valor);
```

13. Probar @Output()

El `@Output` es un `EventEmitter` observable → te suscribes directamente sin necesitar componente padre.

```
const recibidos: any[] = [];
comp.miOutput.subscribe(v => recibidos.push(v));
```

```
el.querySelector('button')!.click();

expect(recibidos).toEqual([valorEsperado]);
```

14. Probar métodos privados o protected

TypeScript no permite acceder a privados directamente, pero en tests puedes usar corchetes:

```
expect(comp['metodoPrivado']()).toBe('algo');
expect(comp['propiedadProtected']).toBe(42);
```

15. Simular clicks

```
const button = el.querySelector('button.btn-test') as HTMLButtonElement;
button.click();
fixture.detectChanges(); // actualiza el DOM después del click
expect(el.querySelector('.resultado')?.textContent).toBe('1');
```

16. Factory functions

Función que crea objetos de prueba reutilizables. Evita repetir el objeto entero en cada test.

```
// Fuera del describe (o dentro, ambos funcionan)
function makeApp(id: number, assignedDay?: WeekDay): InterfaceHouseForm {
  return {
    id,
    firstName: `User${id}`,
    lastName: 'Test',
    email: `user${id}@test.com`,
    housingLocationId: 1,
    consultaDate: new Date(2026, 2, 6),
    assignedDay,
  };
}

// Uso en los tests:
service.addAppointment(makeApp(1));
service.addAppointment(makeApp(2, WeekDay.Monday));
```

Ventaja: si la interfaz añade un campo nuevo, solo tocas la factory.

17. Tests de servicios con localStorage

```
beforeEach(() => {
  localStorage.clear(); // limpia el storage real (JSDOM)
  TestBed.configureTestingModule({
    providers: [MiServicio, LocalstorageService],
  });
  localStorageService = TestBed.inject(LocalstorageService);
  service = TestBed.inject(MiServicio);
  localStorageService.setAllApplications([]); // limpia el estado del servicio
  service.clear(); // limpia estado interno
});
```

Hay que limpiar en **dos sitios**: el `localStorage` global del DOM y el estado interno del servicio.

18. Spies — vi.spyOn

Un spy **observa** un método sin reemplazar su código real.

```
const spy = vi.spyOn(localStorageService, 'getAllApplications');

service.load(); // getAllApplications real se ejecuta

expect(spy).toHaveBeenCalledTimes(1); // ¿se llamó?
expect(spy).toHaveBeenCalledWith(datos); // ¿con qué argumentos?
```

Cuándo usarlo: cuando quieres verificar que tu código **hace las llamadas correctas** a sus dependencias.

19. Mocks — mockReturnValue / mockResolvedValue

El spy **reemplaza** lo que devuelve el método. El código real NO se ejecuta.

Síncrono:

```
vi.spyOn(service, 'getAllApplications').mockReturnValue([makeApp(1)]);
```

Async (Promises):

```
// Éxito
vi.spyOn(apiService, 'getRandomMeal').mockResolvedValue({ idMeal: 1, ... });

// Fallo
```

```
vi.spyOn(apiService, 'getRandomMeal').mockRejectedValue(new Error('Sin conexión'));
```

Cuándo usar mocks:

- La dependencia es lenta (API, base de datos)
- Quieres simular errores (500, red caída, respuesta null)
- Quieres aislar completamente el servicio bajo prueba
- La dependencia aún no está implementada

20. Mockear fetch global

`fetch` no es un servicio Angular inyectado — es una función global. Se mockea así:

```
// Helper reutilizable
function mockFetchOk(body: object): Promise<Response> {
  return Promise.resolve({
    ok: true,
    json: () => Promise.resolve(body),
  } as Response);
}

function mockFetchError(status = 500): Promise<Response> {
  return Promise.resolve({
    ok: false,
    status,
    json: () => Promise.resolve({}),
  } as Response);
}

// En el test:
vi.spyOn(globalThis, 'fetch').mockReturnValue(mockFetchOk({ meals: [rawMeal] }));
const result = await service.getRandomMeal();
expect(result.idMeal).toBe(52772);
```

Lo que probamos con esto:

- Que se llama a `fetch`
- Que el JSON se procesa correctamente
- Que se devuelve `[]` cuando `meals` es `null`
- Que se maneja el error HTTP correctamente

21. afterEach — restaurar spies

```
afterEach(() => vi.restoreAllMocks());
```


Sin esto, un spy del test 5 contamina el test 6. Siempre en el **describe** principal.

22. Cobertura de código

```
ng test --coverage
```

Genera un informe HTML en **coverage/**. Tipos de cobertura:

Tipo	Qué mide
Statements	Líneas ejecutadas
Branches	Caminos de if/else , && , , bucles
Functions	Métodos llamados
Lines	Líneas físicas ejecutadas

Ramas (branches): cada **if** tiene 2 ramas (true/false). Si solo pruebas una, el % baja.

```
if (!response.ok) { // rama 1: ok=false → throw
  throw new Error()
}
// rama 2: ok=true → continúa
```

Alta cobertura ≠ buenos tests. Es orientación, no objetivo final.

23. Resumen visual — ¿qué usar cuándo?

Quiero...	Uso
Probar una clase pura (sin Angular)	new MiClase() directamente
Probar un servicio Angular	TestBed + inject
Probar un componente	TestBed.createComponent + fixture
Acceder al DOM del componente	fixture.nativeElement + detectChanges()
Verificar que se llamó a un método	vi.spyOn + toHaveBeenCalled
Controlar qué devuelve una dependencia	vi.spyOn + mockReturnValue
Simular una API async que funciona	mockResolvedValue
Simular una API async que falla	mockRejectedValue
Simular fetch global	vi.spyOn(globalThis, 'fetch')
Probar @Input	fixture.componentRef.setInput

Quiero...	Uso
Probar @Output	<code>.subscribe()</code> sobre el EventEmitter
Probar método privado	<code>comp['metodoPrivado']()</code>